Labo 1 - Van vanilla JavaScript tot Alpine

BIJ	hoofdstuk 1 (1.2, 1.4, 1.5, 1.9, 1.10)
DUUR	1 labosessie; oefening 4 en 5 mag je thuis afwerken
AI	zonder AI aanbevolen; oefening 2 zonder AI; oefening 5 AI als debughulp
DOOR	Sven Mariën, Kristof Michiels en Andie Similon
CONTEXT	een takenlijst, drie keer gebouwd

Dit vak heet Web Frameworks. In dit eerste labo gaan we nog bouwen zonder Angular. Je bouwt dezelfde kleine applicatie, een takenlijst. Eerst in vanilla JavaScript, waar jij zonder framework alles doet. Dan met een reactief systeem dat je zelf schrijft in ca. twintig regels. Dan in Alpine.js. Aan het einde zorg je voor vier werkende mappen, een `README.md` met je antwoorden, en een computer die klaar is om vanaf volgende week met Angular te werken.

We gaan er voor dit labo van uit dat:

- je je vaardigheden verworven in Web Technology en Web Programming nog niet bent verleerd.
- je de syllabus hoofdstuk 1 hebt gelezen en verwerkt. Indien niet doe dit vooraleer je aan het labo begint.

Na dit labo kan je:

- het synchronisatieprobleem van handmatige DOM-scripting benoemen en aanwijzen in code
- een minimaal reactief systeem (`signal` en `effect`) schrijven en gebruiken
- dezelfde UI-concepten (state, two-way binding, events, lussen, condities) herkennen in vanilla en in Alpine
- een omgeving voor Angular opzetten en controleren

AI in dit labo

Oefening 1, 3 en 4 zijn **zonder AI aanbevolen**. Dit zijn de fundamenten voor het werk van het hele semester voor dit vak, en dat werk moet je zelf doen. Oefening 2 maak je **zonder AI**: fouten vinden in de code van iemand anders is exact wat je op de vaardigheidstoets ook zonder AI moet kunnen. Bij oefening 5 mag AI als debughulp voor je installatie: bvb een foutmelding van npm of de CLI laten verklaren. Een **concept laten uitleggen mag altijd**. Elke regel code die je aflevert, moet je in principe mondeling kunnen uitleggen.

Voor je begint

Download `labo-01-starter.zip` en pak het uit. Je krijgt de mappen `opdracht1` tot `opdracht4`. Er is geen build step: open een map in VS Code en start `index.html` met Live Server, zoals vorig jaar. Maak in de hoofdmap een `README.md` voor je antwoorden.

1. De takenlijst in vanilla

Open `opdracht1/` en lees `app.js`. Eén array `taken` is de state. Eén functie `render()` bouwt de lijst op. Elke event handler doet twee dingen: de data aanpassen en `render()` aanroepen (syllabus 1.2).

Breid uit, in deze volgorde:

- een teller "Nog X open" boven de lijst (het element `#teller` staat klaar); hij klopt na toevoegen, afvinken en verwijderen
- drie filterknoppen: Alle, Open, Gedaan; de lijst toont alleen de gekozen soort, de teller blijft over alle taken gaan

Tel daarna: op hoeveel plaatsen in je code roep je zelf `render()` of een teller-update aan?

Reflectie (README): noteer dat getal. Wat gebeurt er als een collega later een nieuwe mutatie toevoegt en één aanroep vergeet? Hoe zou je dat als tester merken?

2. Bug hunt

In `opdracht2/` staat een "afgewerkte" versie van dezelfde app, met teller en filter, ingeleverd door een fictieve student die het synchronisatieprobleem onderschatte. Er zitten **drie fouten** in met hetzelfde soort symptoom: data en scherm lopen uit elkaar, zonder foutmelding in de console.

Vind er minstens twee. Verbeter de code.

Tip: de console helpt je hier niet, want er is niets "stuk". Gebruik de app zoals een gebruiker en vergelijk wat je ziet met wat er in de data moet staan.

Reflectie (README): per gevonden fout het symptoom (welke handeling toont verkeerde informatie), de oorzaak (welke regel, wat ontbreekt) en de remedie.

3. Bouw je eigen reactiviteit

Dit is het hart van het labo. In `opdracht3/` staat `signal.js`: het reactieve systeem uit syllabus 1.4, met drie gaten (TODO 1 tot 3). De bijgeleverde `app.js` gebruikt het al: één `effect()` dat de lijst rendert, één `effect()` voor de teller, en handlers die alleen nog `set()` aanroepen. Zolang jouw TODO's openstaan, doet de app niets.

- werk `signal.js` af; probeer het eerst zelf en neem pas daarna de syllabus erbij
- open de app: kloppen je TODO's, dan werkt alles in één keer; test toevoegen, afvinken, verwijderen en filteren
- zoek in `app.js` naar handmatige render-aanroepen; het zijn er nul
- bekijk hoe de handlers de lijst wijzigen: nooit `push` op de bestaande array, altijd `set(...)` met een nieuwe array (`[...lijst, item]`, `map`, `filter`)

Reflectie (README): waarom moet dat laatste zo? En: jouw `signal.js` telt twintig regels, Angular enkele honderdduizenden. Noem drie dingen die een echt framework méér doet dan jouw versie (syllabus 1.4).

4. Dezelfde app in Alpine

Alpine ken je niet, en dat is het punt. Een library of framework dat je nog nooit zag, komt in je loopbaan om de paar maanden op je bureau. Zo vind je er in een kwartier je weg in:

- zoek eerst wat het oplost en waar het staat op het spectrum (syllabus 1.5); bij Alpine: gedrag toevoegen aan HTML die er al is
- ga naar de officiële documentatie, niet naar een tutorial of een model: alpinejs.dev/start-here. Kijk naar de datum en het versienummer; dit labo gebruikt Alpine 3
- typ het kleinste werkende voorbeeld over en verander er iets aan, voor je aan je eigen opdracht begint
- lees daarna niet alles, maar zoek per probleem de ene directive die je nodig hebt; de lijst staat links in de documentatie onder *Directives*

Het kleinste werkende voorbeeld. Typ het in een leeg HTML-bestand, met de script-tag uit `opdracht4/index.html` in de `<head>`:

```
<div x-data="{ aantal: 0, naam: '' }">
  <input x-model="naam" placeholder="Je naam">
  <button @click="aantal++">Klik</button>
  <p x-text="`${naam} klikte ${aantal} keer`"></p>
  <p x-show="aantal >= 5">Dat volstaat.</p>
</div>
```

Alles wat Alpine doet, staat in attributen. Tussen de aanhalingstekens staat gewoon JavaScript, dat aan de data van `x-data` kan.

DIRECTIVE	WAT HET DOET	IN VANILLA DEED JE
`x-data="{ ... }"`	de state van dit stuk pagina; geldt voor alles binnen het element	een variabele bovenaan `app.js`
`x-text="..."`	zet de uitkomst van de expressie als tekst in het element	`element.textContent = ...`
`x-model="veld"`	koppelt een invoerveld in twee richtingen aan een veld van de data	`invoer.value` lezen en terugzetten
`@click="..."`	voert de expressie uit bij een klik; korte vorm van `x-on:click`	`addEventListener('click', ...)`
`x-for="taak in taken"`	herhaalt per item; staat op een `<template>` met precies één element erin, liefst met `:key`	`for`-lus met `createElement`
`x-show="..."`	toont of verbergt het element	`element.hidden = ...`
`:class="..."`	zet een attribuut op basis van een expressie; korte vorm van `x-bind:class`	`classList.toggle(...)`

Let op: Alpine volgt ook mutaties. `taken.push(...)` werkt hier dus wel, anders dan in je eigen `signal.js`. Onthoud dat verschil voor de reflectie van oefening 3, en voor hoofdstuk 2: Angular-signals doen het zoals jouw versie.

Nu de opdracht. In `opdracht4/` staat een HTML-bestand met de Alpine-CDN en een half afgewerkte structuur (TODO 1 tot 7). Werk af met de directives uit de tabel: `x-model` op het invoerveld, `@click` op de knoppen, `x-for` voor de lijst, `x-show` voor de melding bij een lege lijst, en een teller als expressie. Je schrijft geen enkel eigen JavaScript-bestand.

Reflectie (README): (a) welke drie dingen uit je vanilla-versie zijn hier *declaratief* geworden? (b) wat mis je zodra deze app tien schermen en drie developers krijgt? (c) waarom zou HTMX (syllabus 1.5) voor deze app een vreemde keuze zijn? (d) welke van de vier stappen hierboven hielp je het meest, en waar in de documentatie vond je `x-for`?

5. Je omgeving voor labo 2

Volgende week werk je een volledig labo met de Angular CLI. Zorg dat de installatie dan achter de rug is (syllabus 1.9 en 1.10):

- controleer of installeer Node: de huidige LTS-versie (Node 24) via nodejs.org
- installeer de Angular CLI: `npm install -g @angular/cli`
- installeer in VS Code de extensie *Angular Language Service*
- controleer met `node -v` en `ng version`, en zet de uitvoer van beide in je README

Loop je vast, zoek dan eerst je foutmelding in de veelgemaakte fouten van hoofdstuk 1 (F1.1 tot F1.3). Lukt het nog niet, breng dan de exacte foutmelding mee naar labo 2.

Werkt het? Een proefproject

Een installatie is pas klaar als ze iets gebouwd heeft. Genereer één wegwerpproject en doe er vijf kleine experimenten mee. Je moet nog niets begrijpen van wat je ziet; dat is labo 2. Je leert hier één reflex: **voorspel, verander, kijk.**

```
ng new proef
cd proef
ng serve --open
```

Kies CSS, geen server-side rendering en geen configuratie voor AI-tools (syllabus 1.10). Zie je de welkomstpagina van Angular op `localhost:4200`, dan werkt je omgeving. Laat de server draaien en open de map `proef` in VS Code.

Schrijf bij elk experiment eerst op wat je verwacht, in één zin. Verander dan, bewaar, en kijk.

#	BESTAND	VERANDER DIT	KIJK HIER
1	`src/app/app.ts`	de tekst in `signal('proef')` wordt je eigen naam	de pagina
2	`src/index.html`	de inhoud van `<title>`	het tabblad van de browser
3	`src/styles.css`	`body { background: mistyrose; }`	de pagina
4	`src/app/app.html`	zoek `Hello,` en zet er een eigen zin achter	de pagina, en de terminal
5	`src/app/app.ts`	`selector: 'app-root'` wordt `selector: 'app-start'`	de pagina, en de console van DevTools

Na experiment 5 is je pagina leeg. Lees de foutmelding in de console, zoek in `src/index.html` waarom, en herstel op de manier die jou het best lijkt. Stop daarna de server met Ctrl-C. De map `proef` mag weg na labo 2.

Reflectie (README): noteer per experiment je voorspelling en wat er gebeurde. Bij welk experiment zat je ernaast? Wat leert experiment 5 je over het verband tussen `index.html` en `app.ts`?

Klaar?

- oefening 1 tot 4 werken, elk in hun eigen map
- in `opdracht3/app.js` staat geen enkele handmatige render-aanroep
- je README bevat alle reflecties, de uitvoer van `node -v` en `ng version`, en je vijf voorspellingen
- het proefproject draait, en experiment 5 is hersteld

- inleveren via de betreffende opdracht in Digitap.

Thuis

Werk oefening 4 en 5 af als dat in de sessie niet lukte. Lees voor labo 2 syllabus 1.10 volledig: het is de handleiding bij wat je volgende week doet. Er is in week 2 geen theorieles; hoofdstuk 2 volgt in week 3.